AI/ML Track

Interview Preparation Series • Professional Edition

Classic ML Algorithms

02

Compiled with LaTeX • June 15, 2026

harsha.somisetty01@gmail.com

In This Edition

> Every classic ML algorithm is an answer to the same question: *given past observations, what is the best bet about the future?* — they differ only in what ”best” and ”bet” mean.

1 1. Overview --- What it is

Classic ML algorithms are **mathematically grounded, interpretable models** that learn patterns from structured/tabular data without requiring deep neural networks. They span supervised learning (regression, classification), unsupervised learning (clustering, dimensionality reduction), and ensemble methods that combine weak learners into strong ones.

Family	Algorithms
Linear Models	Linear Regression, Logistic Regression, SVM
Tree-Based	Decision Trees, Random Forests, Gradient Boosting (GBM/XGBoost/LightGBM)
Instance-Based	k-Nearest Neighbors
Probabilistic	Naive Bayes
Clustering	k-Means, Hierarchical, DBSCAN
Dimensionality Reduction	PCA, t-SNE, UMAP
Ensembles	Bagging, Boosting, Stacking

Scope of this document: First-principles mastery of each family — the math that drives it, when to reach for it, its failure modes, and exactly how FAANG interviews probe it.

2 2. Why It Exists

Neural networks solve everything, right? Wrong — at least in practice:

- **Tabular data is still king in industry.** Most business data (clickstream, transactions, sensor logs) lives in tables. Tree-based models dominate Kaggle tabular competitions and production ML at FAANG.
- **Interpretability requirements.** Regulated industries (finance, healthcare) require explainable decisions. Logistic Regression and Decision Trees provide direct feature attribution.
- **Speed and simplicity.** A Logistic Regression trains in seconds, deploys in microseconds, and never crashes a TPU pod.
- **Baselines.** Every ML project should start with a linear model baseline. If a neural network can't beat Ridge Regression on your dataset, you have a data problem, not a model problem.
- **Feature engineering lever.** Classic models expose *what features matter* — giving feedback loops that improve data pipelines, which ultimately improve even neural models.

The historical arc: **OLS (1800s)** → **Logistic Regression (1950s)** → **Perceptron** → **SVMs (1990s)** → **Random Forests (2001)** → **Gradient Boosting (Friedman 1999, XGBoost 2016)** → **Transformers (2017)**. Each generation solved a failure mode of the previous one.

3 3. Why FAANG Cares

3.1 Google / YouTube

- **Gradient Boosting** powers YouTube recommendation ranking (the "scoring" layer after candidate generation). LightGBM scores 100M+ candidates per second in ranking pipelines.
- **Logistic Regression** with hand-crafted features was the backbone of Google's ad click-through rate (CTR) prediction for years (FTRL-Proximal algorithm, Google 2013 paper).

3.2 Meta

- **GBDT + Neural Net hybrid (GBDT-NN):** Meta's ad ranking uses gradient boosted trees to generate transformed features that feed into a neural network — the trees capture non-linear feature interactions, the net captures higher-order patterns.
- **k-Means:** User clustering for audience segmentation and look-alike modeling.

3.3 Amazon

- **Random Forests** in fraud detection (interpretable feature importance for chargebacks).
- **k-NN** in product recommendation ("customers who bought X also bought Y" — literally nearest neighbors in embedding space).

3.4 Apple

- **Naive Bayes** still used in spam filtering (iCloud Mail). Fast, privacy-preserving (can train on-device).
- **PCA** for face ID feature compression before downstream classifiers.

3.5 Microsoft (Azure ML / LinkedIn)

- **SVM** historically used in NLP classification tasks (text SVMs with TF-IDF features).
- **LightGBM** is a Microsoft Research product — used across Bing ranking.

Key interview insight: FAANG doesn't ask "which algorithm is best?" — they ask "which algorithm is right *for this constraint set*?" Knowing where each lives in production is what separates senior candidates.

4 4. Core Concepts

4.1 4.1 Linear Regression

Intuition

Fit the **best straight line** (hyperplane in higher dimensions) through data points to predict a continuous output. "Best" = minimizing squared residuals.

The Math

Model: `y_hat = w^T x + b` where `w` = weights, `x` = features, `b` = bias

Loss (OLS — Ordinary Least Squares):

```
L(w) = (1/n) * sum[(y_i - w^T x_i)^2]
     = (1/n) * ||y - Xw||^2
```

Closed-form solution (Normal Equation):

```python
w* = (X^T X)^{-1} X^T y
```

- Works when `X^T X` is invertible (no perfectly collinear features).
- Time complexity: O(n * p^2 + p^3) — cubic in features, so use gradient descent when p > ~10,000.

Gradient descent update:

```
w := w - alpha * (2/n) * X^T(Xw - y)
```

Gauss-Markov Assumptions (OLS is BLUE --- Best Linear Unbiased Estimator)

1. **Linearity:** `y = X*beta + epsilon`
2. **No multicollinearity:** features are not perfectly linearly dependent
3. **Homoscedasticity:** `Var(epsilon_i) = sigma^2` (constant variance)
4. **No autocorrelation:** `Cov(epsilon_i, epsilon_j) = 0` for `i ≠ j`
5. **Exogeneity:** `E[epsilon | X] = 0` (errors uncorrelated with features)
6. *(Optional for inference)* Normality: `epsilon ~ N(0, sigma^2)`

Violation consequences: | Violation | Consequence | Fix | |---|---|---| | Multicollinearity | High variance in w estimates, wrong signs | Ridge/Lasso regularization | | Heteroscedasticity | Inefficient estimates, wrong standard errors | Weighted LS, log-transform y | | Non-linearity | Biased predictions | Polynomial features, tree models | | Autocorrelation | Underestimated variance | Time-series models (ARIMA), GLS |

Regularization

```
Ridge (L2):  L = ||y - Xw||^2 + lambda * ||w||^2       (shrinks all weights)
Lasso (L1):  L = ||y - Xw||^2 + lambda * ||w||_1       (sparsifies weights)
ElasticNet:  L = ||y - Xw||^2 + lambda1*||w||_1 + lambda2*||w||^2
```

Ridge has closed form: `w* = (X^T X + lambda*I)^{-1} X^T y` — the `lambda*I` term fixes invertibility even under multicollinearity.

Lasso encourages sparsity because the L1 ball has corners at the axes — the constrained solution tends to land at corners (where some weights = 0). This is automatic feature selection.

Key hyperparameters: `alpha` (learning rate), `lambda` (regularization strength), number of iterations

Complexity: Training O(n*p^2 + p^3) closed form, O(n*p*iter) gradient descent; Inference O(p)

Strengths: Fast, interpretable, works on very high-dimensional sparse data (with regularization), provides uncertainty estimates (via confidence intervals)

Weaknesses: Assumes linearity, sensitive to outliers (squared loss), needs feature scaling for gradient descent

4.2 4.2 Logistic Regression

Intuition

Model the **probability** that an input belongs to class 1. Use a linear model internally, then squeeze the output into [0,1] with the sigmoid function.

The Math

Sigmoid:

```python
sigma(z) = 1 / (1 + e^{-z})

Properties:
- sigma(0) = 0.5
- sigma(+inf) = 1, sigma(-inf) = 0
- d/dz sigma(z) = sigma(z) * (1 - sigma(z))   <-- elegant gradient
```

Model:

```python
P(y=1 | x) = sigma(w^T x + b)
```

Log-Loss (Binary Cross-Entropy):

```
L(w) = -(1/n) * sum[ y_i * log(p_i) + (1-y_i) * log(1-p_i) ]
```

Why log-loss? It's the **negative log-likelihood** under a Bernoulli distribution. Maximizing likelihood = minimizing log-loss.

No closed form — must use gradient descent:

```python
dL/dw = (1/n) * X^T (p - y)
        where p_i = sigma(w^T x_i)
```

The gradient has the same form as linear regression — clean and memorable.

Decision Boundary

```python
P(y=1|x) ≥ 0.5  when  w^T x + b ≥ 0
```

The boundary is the hyperplane `w^T x + b = 0` — **linear** in feature space. For non-linear boundaries, add polynomial features or use kernels.

Interpretation: Log-Odds (Logit)

```python
log[P(y=1)/(1-P(y=1))] = w^T x + b
```

Each weight `w_j` tells you: "a unit increase in feature `x_j` multiplies the **odds** by `e^{w_j}`." This is why logistic regression is the workhorse in medicine and social science — coefficients have direct interpretations.

Multi-class: Softmax Regression

```python
P(y=k | x) = e^{w_k^T x} / sum_j[e^{w_j^T x}]
Loss: Cross-Entropy = -(1/n) * sum_i sum_k [y_{ik} * log(p_{ik})]
```

Key hyperparameters: Regularization (C = 1/lambda in sklearn), solver (lbfgs, liblinear,

saga), max_iter, class_weight

Strengths: Probabilistic output, fast, interpretable weights, works on high-dim sparse data (text), no feature scaling required (but helps convergence)

Weaknesses: Assumes linear decision boundary, poor with feature interactions unless manually added, sensitive to outlier features

When to use: Binary/multiclass classification, when you need calibrated probabilities, as a baseline, in production when inference speed matters, when interpretability is legally required

4.3 4.3 Decision Trees

Intuition

Recursively **split** the data using the feature that best separates classes/reduces prediction error. Creates a tree where each internal node is a test, each leaf is a prediction.

Splitting Criteria

For Classification:

Entropy (Information Gain):

```python
Entropy(S) = -sum_k [ p_k * log2(p_k) ]

Information Gain(S, A) = Entropy(S) - sum_{v in vals(A)} [ |S_v|/|S| * Entropy(S_v) ]
```

- Entropy = 0 when all samples are same class (pure)
- Entropy = 1 for binary with 50/50 split (maximum impurity)

Gini Impurity:

```
Gini(S) = 1 - sum_k [ p_k^2 ]
        = sum_k [ p_k * (1 - p_k) ]
```

- Gini = 0 when pure, Gini = 0.5 for 50/50 binary split
- **Gini is computationally cheaper** (no log), used in sklearn by default
- **Entropy is more sensitive** to class probability changes — theoretically better for imbalanced problems

Mathematically: Gini approximates entropy via `ln(x) ≈ (x-1)`. In practice, results are nearly identical.

For Regression:

```
MSE split: choose feature/threshold minimizing sum of within-child MSE
Variance reduction: Var(parent) - weighted_avg[Var(children)]
```

The Splitting Algorithm (CART)

```javascript
function BuildTree(data, depth):
    if stopping_criterion_met:
        return Leaf(majority_class or mean(y))

    best_feature, best_threshold = None, None
    best_gain = -inf

    for each feature f:
```

```
        for each candidate threshold t:
            gain = impurity(data) - weighted_impurity(split(data, f, t))
            if gain > best_gain:
                best_gain = gain
                best_feature, best_threshold = f, t

    left, right = split(data, best_feature, best_threshold)
    return Node(best_feature, best_threshold,
                left=BuildTree(left, depth+1),
                right=BuildTree(right, depth+1))
```

Candidate thresholds: Sort feature values, try midpoints between consecutive unique values. O(n log n) per feature.

Total training complexity: O(n * p * log(n) * depth) per node, roughly O(n * p * log^2(n)) total

Pruning

Pre-pruning (early stopping):

- `max_depth`: limit tree depth
- `min_samples_split`: minimum samples to split a node
- `min_samples_leaf`: minimum samples in a leaf
- `min_impurity_decrease`: only split if gain exceeds threshold

Post-pruning (cost-complexity pruning):

```
Minimize: sum_leaves[impurity(leaf)] + alpha * num_leaves
```

- Grow full tree, then prune leaves bottom-up
- `alpha` (ccp_alpha in sklearn) controls complexity penalty
- Cross-validate to find best alpha

Strengths

- Interpretable (can print the tree, generate rules)
- Handles mixed feature types naturally
- Handles missing values (surrogate splits)
- No feature scaling needed
- Captures non-linear interactions

Weaknesses

- **High variance** — small data changes create very different trees (instability)
- **Greedy** — local splits are not globally optimal
- Tends to overfit without pruning
- Biased toward high-cardinality features (Gini handles this better than IG)
- **Piecewise constant prediction** — bad at smooth regression

Key hyperparameters: max_depth, min_samples_split, min_samples_leaf, criterion (gini/entropy), ccp_alpha

4.4 4.4 Random Forests

Intuition

Train **many decision trees on random subsets of data and features**, then aggregate their predictions. The randomness decorrelates the trees — their errors cancel out.

The Algorithm: Bagging + Feature Subsampling

```javascript
function RandomForest(X, y, n_trees, max_features):
    trees = []
    for i in 1..n_trees:
        # Bootstrap sample
        X_boot, y_boot = bootstrap(X, y)          # sample n rows WITH replacement

        # Build tree with feature subsampling
        tree = BuildTree(X_boot, y_boot,
                         max_features=sqrt(p))     # at each split, consider only sqrt(p) features
        trees.append(tree)

    return trees

function Predict(x, trees):
    predictions = [tree.predict(x) for tree in trees]
    return majority_vote(predictions)              # or mean() for regression
```

Why Does This Work?

Bias-Variance Tradeoff:

```
E[(y - y_hat)^2] = Bias^2 + Variance + Irreducible_Noise

Single deep tree:  Low Bias,  HIGH Variance  (overfits)
Random Forest:     Low Bias,  Low Variance   (errors average out)
```

Mathematical intuition: If we have `B` trees each with variance `sigma^2` and pairwise correlation `rho`:

```
Var(average) = rho * sigma^2 + (1-rho)/B * sigma^2
```

- If trees were identical (rho=1): no variance reduction
- If trees were independent (rho=0): variance reduces by 1/B
- Random feature subsampling **reduces rho** without increasing bias much

Out-Of-Bag (OOB) Error

Each bootstrap sample leaves out ~36.8% of data (`(1-1/n)^n → 1/e`). These OOB samples form a free validation set:

- OOB error ≈ test error without needing a held-out set
- Useful when data is scarce

Feature Importance

```
Importance(feature_j) = sum_trees sum_nodes_using_j [
    weighted_impurity_decrease(node)  # weighted by n_samples
] / n_trees
```

Mean Decrease in Impurity (MDI) — fast but biased toward high-cardinality features. **Per-**

mutation Importance — shuffle feature j in validation set, measure accuracy drop. Slower but unbiased.

Key hyperparameters: n_estimators, max_features (sqrt(p) for classification, p/3 for regression), max_depth, min_samples_leaf, bootstrap, oob_score

Complexity: Training O(B * n * sqrt(p) * log^2(n)), Inference O(B * depth)

Strengths: Robust to overfitting, handles high-dim data, feature importance, OOB error, parallel training, handles missing values reasonably, works well out-of-the-box

Weaknesses: Less interpretable than single tree, slow inference for very large forests, memory-intensive, can overfit on noisy datasets, not great at extrapolation

4.5 4.5 Gradient Boosting (GBM / XGBoost / LightGBM)

Intuition

Train trees **sequentially**, where each tree corrects the errors (residuals) of the previous ensemble. Gradient boosting generalizes this: each tree fits the **negative gradient** of the loss function.

The Core Algorithm

```python
Initialize: F_0(x) = argmin_gamma sum_i L(y_i, gamma)
            (e.g., mean(y) for MSE, log(p/(1-p)) for log-loss)

For m = 1 to M:
    1. Compute pseudo-residuals (negative gradient):
       r_{im} = -[dL(y_i, F(x_i)) / dF(x_i)]    for all i

       For MSE: r_{im} = y_i - F_{m-1}(x_i)      (just the residuals!)
       For log-loss: r_{im} = y_i - sigma(F_{m-1}(x_i))

    2. Fit a regression tree h_m(x) to {(x_i, r_{im})}

    3. Find optimal step size:
       gamma_m = argmin_gamma sum_i L(y_i, F_{m-1}(x_i) + gamma * h_m(x_i))

    4. Update:
       F_m(x) = F_{m-1}(x) + learning_rate * gamma_m * h_m(x)

Final: F_M(x)
```

Key insight: We're doing **gradient descent in function space** — instead of updating weights, we're updating the entire function by adding a new tree in the direction that reduces loss.

Why Gradient, Not Just Residuals?

For MSE loss, `dL/dF = -(y - F(x))` — the negative gradient IS the residuals. For other losses (MAE, log-loss, Huber), the "residuals" change, but the gradient descent framework handles them uniformly.

XGBoost Improvements Over Vanilla GBM

1. Regularized objective:

```python
Obj = sum_i L(y_i, y_hat_i) + sum_k Omega(f_k)
Omega(f) = gamma*T + (1/2)*lambda*||w||^2
```

Where T = number of leaves, w = leaf weights. This directly penalizes tree complexity.

2. Approximate greedy algorithm: Instead of trying all split points, use quantile sketches to find split candidates. Much faster for large datasets.

3. Sparsity-aware split finding: Learn default direction for missing values — handles sparse features efficiently.

4. Column block for parallel learning: Pre-sort features, store in compressed column blocks. Enables parallel split finding across features.

5. Cache-aware access and out-of-core computation: Designed for data that doesn't fit in RAM.

LightGBM Improvements Over XGBoost

1. Gradient-based One-Side Sampling (GOSS): Keep all large-gradient samples (they have high information), randomly downsample small-gradient samples. Faster with minimal accuracy loss.

2. Exclusive Feature Bundling (EFB): Bundle mutually exclusive sparse features into one dense feature. Reduces effective number of features.

3. Leaf-wise tree growth (vs level-wise in XGBoost):

```
Level-wise (XGBoost):        Leaf-wise (LightGBM):
Level 0:  [root]             Always split the leaf with max delta_loss
Level 1:  [L] [R]            Can create deep, asymmetric trees
Level 2:  [LL][LR][RL][RR]   More accurate but can overfit
```

Leaf-wise is faster (fewer splits to get same accuracy) but needs `num_leaves` control.

4. Histogram-based splitting: Bin continuous features into 255 buckets. Huge memory and speed win.

Comparison Table

Property	GBM	XGBoost	LightGBM	CatBoost
Speed	Slow	Medium	Fast	Medium
Memory	High	Medium	Low	Medium
GPU support	No	Yes	Yes	Yes
Categorical features	Manual encoding	Manual encoding	Native (limited)	Native (excellent)
Tree growth	Level-wise	Level-wise	Leaf-wise	Symmetric
Regularization	Shrinkage	L1, L2, tree complexity	L1, L2	Leaf-wise + ordered boosting
Missing values	Manual	Native	Native	Native
Best for	Baseline	General	Large datasets	Categorical-heavy

Key hyperparameters:

- `n_estimators` (M): number of trees — more = less bias, more variance, slower
- `learning_rate` (eta): step size — smaller = more robust, needs more trees
- `max_depth`: tree depth — shallower = less variance, more bias
- `subsample`: row subsampling per tree (stochastic GBM) — reduces variance
- `colsample_bytree`: column subsampling — like RF feature randomness
- `min_child_weight` / `min_child_samples`: regularization on leaves
- `lambda/alpha`: L2/L1 regularization on leaf weights

Golden rule: `learning_rate * n_estimators` should be roughly constant. Lower LR + more trees = better but slower.

Complexity: Training O(M * n * p * log(n)), Inference O(M * depth)

Strengths: State-of-the-art on tabular data, handles heterogeneous features, built-in regularization, feature importance, handles missing values, early stopping

Weaknesses: Many hyperparameters, sequential training (harder to parallelize than RF), can overfit with wrong params, slow to train at scale, requires careful tuning

4.6 4.6 Support Vector Machines (SVM)

Intuition

Find the **hyperplane that maximizes the margin** between classes. The margin is the distance from the hyperplane to the nearest training point of each class. Wider margin = better generalization.

Hard-Margin SVM (linearly separable)

Objective:

```python
Maximize margin = 2 / ||w||
Subject to: y_i(w^T x_i + b) ⩾ 1   for all i
Equivalently: Minimize (1/2)||w||^2
              Subject to: y_i(w^T x_i + b) ⩾ 1
```

The hyperplane is `w^T x + b = 0`.
The two margin planes are `w^T x + b = +1` and `w^T x + b = -1`.
Margin width = `2/||w||`.

Support Vectors: The training points that lie exactly on the margin planes (`y_i(w^T x_i + b) = 1`). Only these points determine the hyperplane — the rest are irrelevant. This is why SVMs are robust to outliers far from the boundary.

Soft-Margin SVM (C-SVM, non-separable)

Allow some points to violate the margin via **slack variables** `xi_i ⩾ 0`:

```python
Minimize: (1/2)||w||^2 + C * sum_i xi_i
Subject to: y_i(w^T x_i + b) ⩾ 1 - xi_i
            xi_i ⩾ 0
```

C = regularization parameter:

- `C → ∞`: Hard margin, no violations allowed, risk of overfitting
- `C → 0`: Wide margin, many violations allowed, underfitting
- **Smaller C = more regularization** (opposite intuition from other models)

Hinge Loss equivalence:

```python
Hinge Loss: max(0, 1 - y_i * f(x_i))
SVM objective = (1/2)||w||^2 + C * sum_i max(0, 1 - y_i * f(x_i))
              = L2 regularization + C * sum of hinge losses
```

The Kernel Trick

Map data to higher dimensions where it becomes linearly separable — without ever computing the high-dimensional vectors.

Key insight: The SVM dual formulation depends on data only through **inner products** `x_i^T x_j`:

```
Dual: Maximize sum_i alpha_i - (1/2) sum_{ij} alpha_i alpha_j y_i y_j (x_i^T x_j)
```

Replace `x_i^T x_j` with `K(x_i, x_j)` — a **kernel function** that computes the inner product in some high/infinite-dimensional space, without going there explicitly.

Common kernels:

Kernel	Formula	When to use
Linear	`K(x,z) = x^T z`	Linearly separable, high-dim sparse (text)
Polynomial	`K(x,z) = (gamma*x^T z + r)^d`	Moderate non-linearity, image features
RBF/Gaussian	‘K(x,z) = exp(-gamma *	
Sigmoid	`K(x,z) = tanh(gamma*x^T z + r)`	Similar to neural net activation, rarely used

RBF Intuition: `K(x,z)` measures similarity — it's 1 when x=z, 0 when they're infinitely far apart. Points ”vote” on classification weighted by similarity. The implicit feature space is **infinite-dimensional**.

The Kernel Trick computation:

```
Normal: compute phi(x) [infinite dim vector], then dot product --> impossible
Kernel: compute K(x,z) = <phi(x), phi(z)> directly --> O(n_features), no expansion needed
```

Key hyperparameters: C, kernel type, gamma (for RBF: smaller gamma = wider RBF, smoother boundary), degree (polynomial), coef0

Complexity: Training O(n^2 * p) to O(n^3 * p), Inference O(n_sv * p) — scales poorly with large n

Strengths: Works in high-dim spaces, effective when n < p, memory efficient (only support vectors stored), kernel trick for non-linear data, global optimum (convex problem)

Weaknesses: Slow on large datasets (O(n^2-3)), no probability output (need Platt scaling), sensitive to feature scaling, hard to interpret, tricky to tune gamma and C

When to use: Small-to-medium dataset, many features (text), non-linear classification with RBF, when you need a clear margin

4.7 4.7 k-Nearest Neighbors (k-NN)

Intuition

To classify a new point, find its **k nearest neighbors** in the training set and take a majority vote (classification) or average (regression).

The simplest possible model — no training, just memorization.

The Algorithm

```
Training: Store all (x_i, y_i)           # O(n*p) space

Prediction for query q:
1. Compute distance(q, x_i) for all i    # O(n*p)
2. Sort by distance                       # O(n log n)
3. Take k smallest                        # O(k)
4. Return majority vote or mean           # O(k)
```

Distance metrics:

```python
Euclidean: d(x,z) = sqrt(sum_j (x_j - z_j)^2)   -- most common
Manhattan: d(x,z) = sum_j |x_j - z_j|            -- robust to outliers
Minkowski: d(x,z) = (sum_j |x_j - z_j|^p)^{1/p} -- generalizes both
Cosine:    d(x,z) = 1 - (x^T z)/(||x||*||z||)   -- text/embedding similarity
Hamming:   for binary/categorical features
```

Choosing k

```python
k=1:       Zero training error, very high variance (memorizes noise)
k=n:       Predict majority class always -- zero variance, high bias
k=sqrt(n): Common heuristic, balance point
```

Small k = high variance (overfitting), Large k = high bias (underfitting)
Use cross-validation to choose k. Odd k avoids ties in binary classification.

Curse of Dimensionality

In high dimensions, **all points become equidistant** — nearest neighbors are no more similar than random points. The ratio of max to min distance converges to 1 as dimensionality grows.

```
d_max / d_min → 1 as p → ∞
```

Consequence: k-NN degrades badly in high dimensions. Always apply PCA/feature selection first.

Improvements

- **KD-tree:** Organize data in a tree for O(p log n) query instead of O(np). Breaks down for p > ~20.
- **Ball-tree:** Better than KD-tree for high dimensions and non-Euclidean metrics.
- **Approximate Nearest Neighbors (ANN):** FAISS, HNSW, Annoy — trade exact for speed at scale.

Key hyperparameters: k, distance metric, weights (uniform vs distance-weighted), algorithm (ball_tree, kd_tree, brute)

Complexity: Training O(n*p) (just storing), Inference O(*n*p + n log n) per query

Strengths: Simple, interpretable, no training, naturally handles multi-class, works well with enough data

Weaknesses: Slow at inference, memory-intensive (stores all data), sensitive to irrelevant features, needs feature scaling, curse of dimensionality

When to use: Small datasets, recommendation systems (in embedding space), anomaly detection, when decision boundary is complex and data is low-dimensional

4.8 4.8 Naive Bayes

Intuition

Apply Bayes' theorem to classify, with the **"naive" assumption that features are conditionally independent** given the class. Despite this usually being wrong, it works surprisingly well.

Bayes' Theorem

```python
P(y=k | x) = P(x | y=k) * P(y=k) / P(x)
           ∝ P(x | y=k) * P(y=k)          # P(x) is constant for all classes
```

The naive assumption:

```python
P(x | y=k) = prod_j P(x_j | y=k)         # features independent given class
```

Classification rule:

```python
y_hat = argmax_k [log P(y=k) + sum_j log P(x_j | y=k)]
```

Log-sum is numerically stable and avoids underflow from multiplying many small probabilities.

Variants

Gaussian NB (continuous features):

```python
P(x_j | y=k) = N(x_j; mu_{jk}, sigma_{jk}^2)
mu_{jk} = mean of feature j in class k
sigma_{jk}^2 = variance of feature j in class k
```

Multinomial NB (count features, text):

```python
P(x_j | y=k) = theta_{jk}^{x_j}      # x_j is count of word j
theta_{jk} = (count of word j in class k + alpha) / (total words in class k + alpha * |V|)
```

`alpha` is Laplace/additive smoothing — prevents zero probabilities for unseen words.

Bernoulli NB (binary features):

```
P(x_j | y=k) = p_{jk}^{x_j} * (1-p_{jk})^{1-x_j}
```

Why It Works Despite Wrong Assumption

- For classification, we only need **ordering** of P(y=k|x) across classes, not exact values
- Independence errors can partially cancel across features
- Works well when features are ”nearly” independent
- Very robust to irrelevant features (they contribute near-uniform P(x_j|y=k))

Key hyperparameters: var_smoothing (Gaussian NB), alpha (Multinomial/Bernoulli NB — Laplace smoothing)

Complexity: Training O(n*p*), *Inference O(p*K) where K = classes

Strengths: Extremely fast, works with tiny datasets, handles high-dim (text) well, good with streaming data, probabilistic output, robust to irrelevant features

Weaknesses: Feature independence assumption violated in most problems, poor probability calibration (use isotonic regression if you need probs), can't learn feature interactions, Gaussian NB assumes normal distribution

When to use: Text classification (spam, sentiment), real-time classification (fast inference), small datasets, streaming/online learning

4.9 4.9 k-Means Clustering

Intuition

Partition n data points into k clusters by assigning each point to its nearest cluster center (centroid), then updating centroids as the mean of assigned points. Iterate until convergence.

Lloyd's Algorithm

```
Initialize: k centroids {mu_1, ..., mu_k} randomly (or k-means++)

Repeat until convergence:
    Assignment step:
        c_i = argmin_k ||x_i - mu_k||^2    for all i

    Update step:
        mu_k = (1/|C_k|) * sum_{i in C_k} x_i    for all k

Converges when assignments stop changing
```

Objective (minimized):

```
J = sum_i ||x_i - mu_{c_i}||^2    (Within-Cluster Sum of Squares, WCSS)
```

This is a non-convex optimization — **local optima exist**. Solution: run multiple random initializations, pick the best.

k-Means++ Initialization

Instead of random initialization, choose centroids with probability proportional to distance from existing centroids:

```
P(x is chosen as next centroid) ∝ min_j ||x - mu_j||^2
```

This spreads initial centroids far apart, typically giving better solutions and faster convergence.

Choosing k: The Elbow Method

```
Plot WCSS vs k:
        WCSS
         |
   High  |*
         | *
         |  *
         |   *----------
         |               (plateau)
    Low  +----------> k
              ^ Elbow = good k
```

The elbow is where adding more clusters has diminishing returns. Formal methods: Silhouette score, Gap statistic, BIC.

Silhouette score:

```
s(i) = (b(i) - a(i)) / max(a(i), b(i))
a(i) = mean intra-cluster distance
b(i) = mean nearest-cluster distance
Range: [-1, 1], higher is better
```

Assumptions and Limitations

- Assumes clusters are **spherical** and roughly **equal size**
- Assumes Euclidean distance is meaningful
- Must specify k in advance
- Sensitive to outliers (use k-Medoids/PAM instead)
- Can fail on ring/crescent/elongated shapes (use DBSCAN/GMM)

Key hyperparameters: n_clusters (k), init (k-means++ or random), n_init (number of restarts), max_iter, random_state

Complexity: Training O(n * k * p * iter), Inference O(k * p) per point

4.10 4.10 Hierarchical Clustering

Intuition

Build a tree of clusters (dendrogram) either **bottom-up (agglomerative)** or top-down (divisive). Cut the tree at the desired level to get clusters.

Agglomerative Clustering

```
Initialize: Each point is its own cluster (n clusters)

Repeat until one cluster remains:
    Find the two closest clusters C_i and C_j
    Merge them into one cluster
    Update distance matrix

Result: Dendrogram (binary tree)
Cut at height h → determines number of clusters
```

Linkage Criteria (how to measure cluster distance)

Linkage	Formula	Behavior
Single	`min d(a,b), a in C_i, b in C_j`	Chaining effect, elongated clusters
Complete	`max d(a,b), a in C_i, b in C_j`	Compact, equal-size clusters
Average	`mean d(a,b), a in C_i, b in C_j`	Balance between single and complete
Ward	Minimize within-cluster variance increase	Usually best, similar to k-means

Ward linkage is the default choice — minimizes the increase in total WCSS when merging, producing compact, similarly-sized clusters.

Complexity: Naive O(n^3), optimized O(n^2 log n) with heap; Inference O(1) after dendrogram built

Strengths: No need to specify k in advance, produces dendrogram (full hierarchy), deterministic, works with any distance metric, can find non-spherical clusters (with single linkage)

Weaknesses: O(n^2) memory for distance matrix, slow for large datasets, can't easily update with new data, cutting the dendrogram is subjective

When to use: When you need a hierarchy, small-to-medium datasets, exploratory analysis, gene expression analysis (bioinformatics standard)

4.11 4.11 DBSCAN

Intuition

Density-Based Spatial Clustering of Applications with Noise. Clusters are regions of high point density. Points in low-density regions are labeled as **noise/outliers**. No need to specify k — clusters emerge from density.

Core Concepts

```python
epsilon (eps): neighborhood radius
min_samples: minimum points to form a dense region

Point types:
- Core point:    has ≥ min_samples points within eps radius (including itself)
- Border point:  within eps of a core point but < min_samples in its own eps
- Noise point:   neither core nor border (outlier)
```

The Algorithm

```javascript
for each unvisited point p:
    neighbors = all points within eps of p

    if |neighbors| < min_samples:
        label p as NOISE
    else:
        create new cluster C
        add p to C
        expandCluster(p, neighbors, C, eps, min_samples)

function expandCluster(p, neighbors, C, eps, min_samples):
    for each q in neighbors:
        if q is NOISE: add q to C (border point)
        if q is unvisited:
            mark q as visited
            q_neighbors = all points within eps of q
```

```
            if |q_neighbors| ≥ min_samples:  # q is also a core point
                neighbors = neighbors U q_neighbors  # expand search
            if q not in any cluster:
                add q to C
```

Key insight: DBSCAN can find **arbitrary-shaped clusters** — it connects core points that are within eps of each other. It naturally handles outliers (noise points).

Choosing eps and min_samples

- **min_samples**: Rule of thumb: `min_samples ≥ dimensionality + 1`, usually 4-10. Higher = less noise sensitivity.
- **eps**: Plot k-distance graph (distance to k-th nearest neighbor for each point), look for the "elbow." The knee of the curve is a good eps.

Complexity: O(n log n) with spatial indexing (kd-tree/ball-tree), O(n^2) naively

Strengths: Finds arbitrary shapes, automatically detects outliers, no need to specify k, robust to different cluster sizes

Weaknesses: Struggles with varying densities (HDBSCAN solves this), sensitive to eps and min_samples, doesn't work well in high dimensions (epsilon loses meaning), not good at classifying new points

When to use: Geospatial clustering, anomaly/outlier detection, when you expect non-spherical clusters, when number of clusters is unknown

4.12 4.12 PCA / Dimensionality Reduction

Intuition

Find the **directions of maximum variance** in the data and project onto fewer dimensions. The first principal component explains the most variance, the second (orthogonal to first) explains the second most, etc.

The Math

Setup: X is n x p, centered (subtract mean from each feature)

Covariance matrix:

```python
Sigma = (1/n) * X^T X     (p x p matrix)
```

Eigenvector decomposition:

```python
Sigma = V Lambda V^T

V = [v_1 | v_2 | ... | v_p]    (eigenvectors = principal component directions)
Lambda = diag(lambda_1, lambda_2, ..., lambda_p)   (eigenvalues, sorted descending)
```

Project onto k components:

```python
Z = X * V_k    (n x k matrix)
V_k = first k columns of V
```

Explained variance ratio:

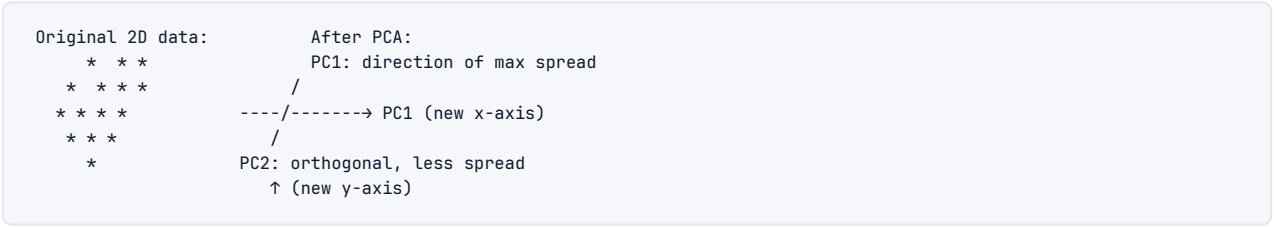
```python
EVR_j = lambda_j / sum_i lambda_i
Cumulative EVR: choose k such that sum_{j=1}^k EVR_j ≥ 0.95 (95% threshold)
```

SVD Connection

```python
X = U Sigma V^T    (SVD)
Principal components = right singular vectors V
Scores = U Sigma = X V
Eigenvalues of covariance = sigma_i^2 / n (squared singular values)
```

In practice, **always use SVD, not eigendecomposition** — more numerically stable, works even when p » n.

Geometric Intuition



The first PC is the axis along which projecting the data retains the most information (minimum reconstruction error).

PCA Assumptions

- Directions of max **variance** = directions of interest (this fails when important patterns are in low-variance directions)
- Linear relationships between features
- Data is centered (subtract mean — PCA is not scale-invariant, must standardize)

Kernel PCA

Apply PCA in the kernel-transformed feature space:

```python
Replace X^T X with kernel matrix K where K_{ij} = k(x_i, x_j)
```

Finds non-linear low-dimensional structure.

t-SNE (brief)

For **visualization** only (2D/3D). Preserves local structure — nearby points in high-dim stay nearby in low-dim. Non-parametric, can't transform new points. Use PCA for preprocessing before t-SNE on large datasets.

UMAP (brief)

Faster than t-SNE, preserves more global structure. Can transform new points. Preferred for large-scale visualization and as a preprocessing step.

Key hyperparameters (PCA): n_components, svd_solver (auto, full, randomized, arpack), whiten

Complexity: O(n * p^2 + p^3) exact, O(n * p * k) randomized SVD
When to use PCA:

- Remove multicollinearity before linear models
- Speed up training (reduce features)
- Visualization (first 2-3 components)
- Noise reduction
- Feature engineering for downstream models

4.13 4.13 Ensemble Methods: Bagging vs Boosting vs Stacking

Bagging (Bootstrap AGGregating)

```
Core idea: Reduce variance by averaging many high-variance models
Training: Independent, parallel -- models don't see each other's errors
Sampling: Bootstrap (with replacement)
Combination: Average (regression), majority vote (classification)
```

```
Data
 |
 +----Bootstrap_1 ---> Model_1 ----+
 |                                 |
 +----Bootstrap_2 ---> Model_2 ----+--> Average/Vote ---> Final Prediction
 |                                 |
 +----Bootstrap_3 ---> Model_3 ----+
```

Examples: Random Forest (bagging + feature randomness), Bagged SVMs
Best for: High-variance, low-bias base models (deep trees)

Boosting

```
Core idea: Reduce bias by sequentially correcting errors
Training: Sequential -- each model focuses on previous model's mistakes
Sampling: Full dataset (or weighted/modified)
Combination: Weighted sum (adaptive weighting)
```

```
Data → Model_1 → Errors
                   |
              Model_2 (focus on errors of Model_1) → Residuals
                                                       |
                                                 Model_3 (focus on Model_2's residuals)
                                                           ...
                                                      Final = weighted sum
```

Examples: AdaBoost, GBM, XGBoost, LightGBM
Best for: Low-variance, high-bias base models (shallow trees, stumps)

Stacking (Stacked Generalization)

```
Core idea: Use a meta-model to optimally combine base model predictions
Training: Two-level training with cross-validation to avoid leakage
```

```
Level 0 (Base models):
  X ---> Model_A (Logistic Regression)  --> pred_A ----+
  X ---> Model_B (Random Forest)        --> pred_B ----+--> [pred_A, pred_B, pred_C]
  X ---> Model_C (XGBoost)              --> pred_C ----+

Level 1 (Meta-model):
```

```
[pred_A, pred_B, pred_C] ---> Meta Learner (often Logistic Regression) ---> Final
```

Stacking implementation (preventing leakage):

```python
# Cross-val approach
oof_predictions = np.zeros((n_train, n_models))
for model_idx, model in enumerate(base_models):
    for fold_idx, (train_idx, val_idx) in enumerate(kfold.split(X)):
        model.fit(X[train_idx], y[train_idx])
        oof_predictions[val_idx, model_idx] = model.predict(X[val_idx])

# Train meta-model on out-of-fold predictions
meta_model.fit(oof_predictions, y)
```

Strengths: Often best accuracy; Weaknesses: complex, slow, overfitting risk if leakage

Comparison Table

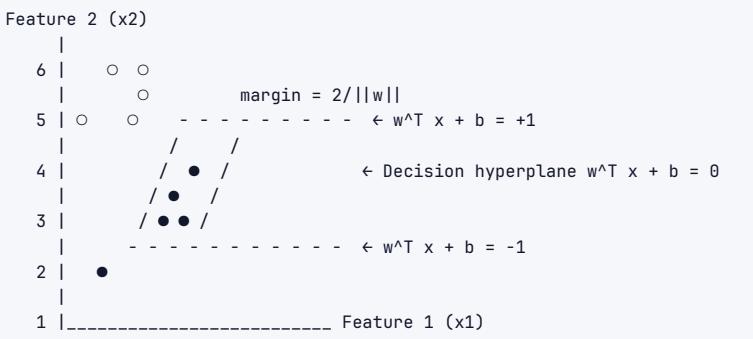
Property	Bagging	Boosting	Stacking
Error type targeted	Variance	Bias	Both
Training order	Parallel	Sequential	Two-stage
Data sampling	Bootstrap	Weighted/full	Full (CV for meta)
Base model	High-variance (deep trees)	High-bias (stumps)	Diverse
Combination	Average/vote	Weighted sum	Meta-model
Speed	Fast (parallel)	Slow (sequential)	Medium
Overfitting risk	Low	Medium (can overfit)	High (if leakage)
Example	Random Forest	XGBoost	Kaggle winning solutions
When to use	Reduce overfitting	Boost a weak model	Maximum accuracy

5 5. Architecture / Diagrams

5.1 Decision Tree Structure

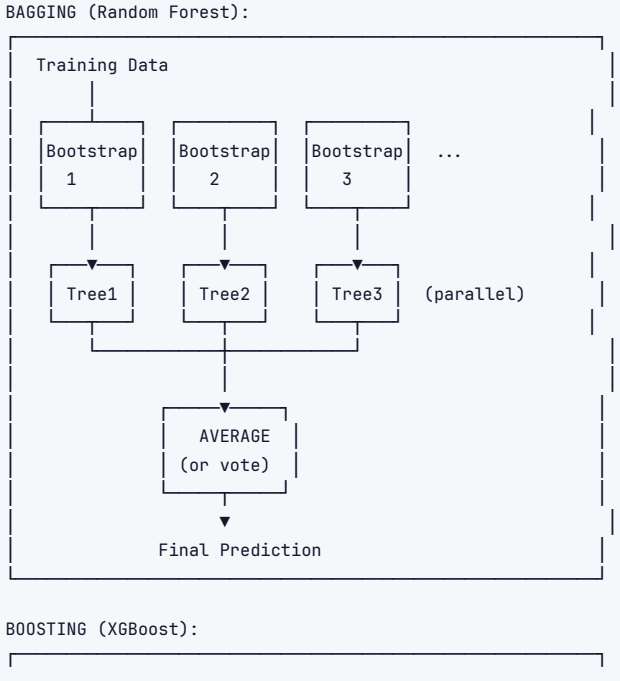
```
                  [Age < 30?]                    ← Root node
                 /           \
            YES /             \ NO
               /               \
       [Income > 50k?]      [Own Home?]           ← Internal nodes
         /       \            /      \
       YES       NO         YES       NO
       /          \         /          \
  [Approve]    [Deny]  [Approve]    [Credit?]     ← Leaves + node
                                      /    \
                                    YES     NO
                                 [Approve] [Deny]

Split criterion: minimize Gini/Entropy
Pruning: remove nodes that don't significantly reduce impurity
```

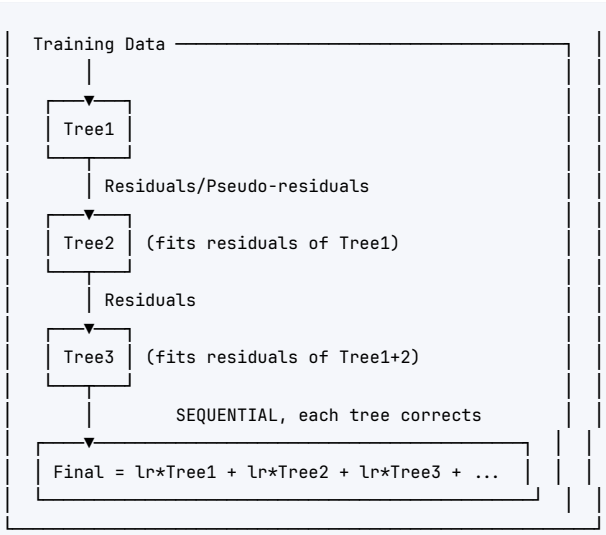
5.2 SVM Maximum Margin Boundary



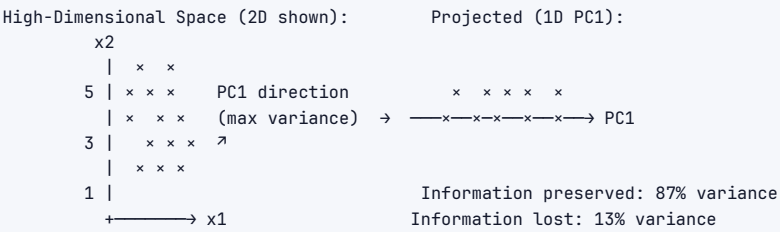
5.3 k-Means Cluster Formation

5.4 Bagging vs Boosting Flow

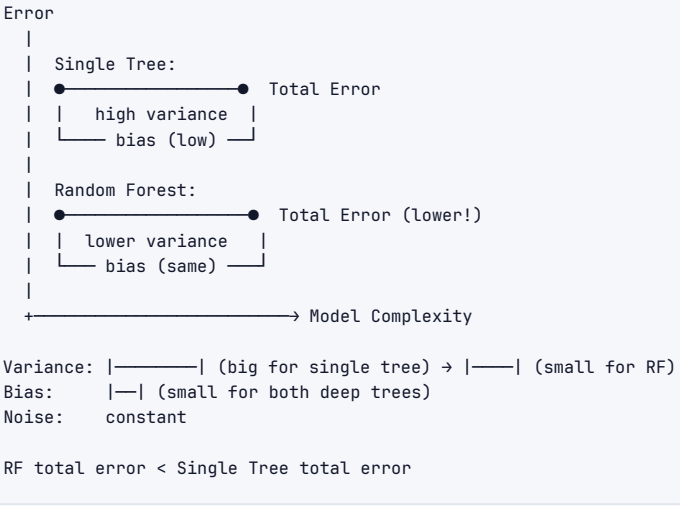




5.5 PCA Projection



5.6 Bias-Variance: Single Tree vs Random Forest



6 6. Real-World Examples

6.1 Linear Regression

- **House price prediction** (Zillow Zestimate started as linear regression with 200+ features)
- **Stock return forecasting** (Fama-French factor models)
- **A/B test analysis** (linear models for CUPED variance reduction at Netflix)

6.2 Logistic Regression

- **Credit scoring** (FICO score model, FDIC regulatory requirement for interpretability)
- **Email spam** (Google's early Spam filter)
- **Google Ads CTR** (billion-scale with FTRL optimization)

6.3 Decision Trees

- **Medical diagnosis protocols** (sepsis screening, CHA2DS2-VASc stroke risk)
- **Customer churn rules** (easily communicated to business stakeholders)
- **Loan approval** (regulatory requirement for explainability in EU AI Act)

6.4 Random Forest

- **Genomics** (predicting gene expression from SNPs — thousands of features)
- **Fraud detection at PayPal** (100+ features, handles missing data naturally)
- **Remote sensing** (classifying satellite pixels into land-use categories)

6.5 Gradient Boosting (XGBoost/LightGBM)

- **Kaggle**: Won >70% of structured data competitions since 2016
- **LinkedIn Feed Ranking**: LightGBM for post relevance scoring
- **Uber surge pricing**: GBM for demand forecasting
- **Microsoft Bing**: LightGBM for document relevance ranking

6.6 SVM

- **OCR (handwriting recognition)** — original MNIST benchmark
- **Bioinformatics**: Protein classification, gene expression microarrays (high-dim, small n)
- **Face detection**: Early face detectors used SVMs with Haar features

6.7 k-NN

- **Netflix/Spotify recommendations** — k-NN in embedding space (collaborative filtering)
- **Medical diagnosis from embeddings**: ”Patients similar to this one had X outcome”
- **Fraud detection**: k-NN anomaly scores

6.8 Naive Bayes

- **Spam filters**: SpamAssassin, iCloud Mail
- **Sentiment analysis**: Fast, surprisingly effective baseline
- **Medical text classification**: On-device, privacy-preserving

6.9 k-Means

- **Customer segmentation**: RFM (Recency, Frequency, Monetary) clusters for marketing
- **Image compression**: Color quantization (k-Means on RGB values → k representative colors)
- **Document clustering**: TF-IDF vectors → topic groups

6.10 PCA

- **Eigenfaces**: Early face recognition (Turk & Pentland, 1991)
- **Risk factor models in finance**: 5-10 PCs explain 70%+ of stock return variance
- **NLP preprocessing**: PCA on TF-IDF matrix before SVM

7 7. Real-Life Analogies

*Imagine a hospital emergency room. A **panel of specialist doctors** uses different methods to diagnose patients — each algorithm is a doctor with a distinct diagnostic style.*

Algorithm	Doctor Analogy	Key Behavior
Decision Tree	The triage nurse with a printed flowchart: "Temp > 38.5°C? → Yes → Chest pain? → No → Rule out pneumonia."	Follows explicit branching rules; fast but rigid; one wrong branch goes down the wrong path
Random Forest	The **medical board**: 100 doctors each examine a random patient subset and a random subset of symptoms; majority vote wins	Cancels out individual doctor errors; much more reliable than any single doctor
Gradient Boosting	The **residency training program**: Intern makes a diagnosis → Senior corrects the mistakes → Fellow corrects remaining errors → Sequential improvement	Each successive doctor specializes in what the previous missed; powerful but takes time to train
SVM	The **surgeon** who draws the clearest anatomical line between healthy and diseased tissue, maximizing the margin for error	Maximizes separation; only cares about boundary cases (support vectors)
k-NN	The doctor who says *"Let me look up the 5 most similar patients we've seen and see what happened to them"*	No model training — pure case-based reasoning; only as good as the patient database
Naive Bayes	The **phone triage doctor** who assumes symptoms are independent: "Fever? +10 pts. Cough? +5 pts. Fatigue? +3 pts."	Fast; wrong about independence (fever and cough often co-occur) but often right enough
Logistic Regression	The **general practitioner** who weighs each symptom linearly: "I assign 2.3× weight to chest pain and 0.7× to age" and gives a probability	Interpretable weights; can explain decision; good calibrated probabilities
k-Means	The **epidemiologist** grouping patients into symptom clusters: "Group A = respiratory, Group B = cardiovascular"	Discovers natural groupings without labels; must specify number of groups upfront
PCA	The **chief diagnostician** who identifies the 3 "vital sign combinations" (principal components) that explain 95% of patient variation	Reduces 50 measurements to 3 key dimensions; loses some detail but gains clarity
DBSCAN	The **outbreak investigator** finding clusters of patients who live near each other + flagging isolated cases as "outliers"	No need to specify clusters; naturally identifies anomalies
Hierarchical Clustering	The **taxonomy specialist** building a family tree of related conditions from common to rare	Produces a full hierarchy; shows how conditions relate at different levels of similarity
Stacking	The **hospital committee**: GP, cardiologist, and pulmonologist each give a diagnosis; a chief of medicine combines their opinions with learned weights	Best accuracy by leveraging diverse specialist views; but complex and slow

The best hospitals (and ML systems) use the right specialist for the right case — and often consult the full board for the hardest diagnoses.

8 8. Memory Tricks / Mnemonics

8.1 Algorithm Selection: "GLDRSNK-BPS" (Great Lakes Don't Run South, Never Kite --- Big Pandas Swim)

- **G**radient Boosting: tabular, winning solution
- **L**ogistic Regression: baseline, interpretable, fast
- **D**ecision Tree: interpretable, rules
- **R**andom Forest: robust, high-dim, OOB
- **S**VM: small n, large p, kernels
- **N**aive Bayes: text, speed, streaming
- **K**-NN: recommendation, embeddings
- **B**oosting (GBM/XGB): sequential correction
- **P**CA: dimensionality, visualization
- **S**tacking: Kaggle, maximum accuracy

8.2 Key Formulas --- SSGH (Signed Symmetrical, Gradient, Hinge)

- **Sigmoid:** "S-shape squeezes infinity into [0,1]"
- **Softmax:** "Sigmoid's multiclass cousin — normalizes exponentials"
- **Gini:** "1 minus sum of squares" → `1 - sum(p^2)`
- **Hinge:** "max(0, 1-y*f(x))" → zero for correct confident predictions

8.3 Bias-Variance Mnemonic: "OVER-UNDER-JUST"

- **Overfit** = High Variance, Low Bias (deep tree, k-NN with k=1)
- **Underfit** = High Bias, Low Variance (linear model on curved data)
- **Just right** = Bagging reduces variance; Boosting reduces bias

8.4 Kernel Trick: "Measure similarity, never transform"

- "You don't need to go to infinite dimensions — just measure how similar points are in that space"
- `K(x,z)` = inner product in feature space without computing feature vectors

8.5 Regularization Directions:

- **Ridge (L2):** "Ridge = Round ball → weights shrink uniformly toward zero, never exactly zero"
- **Lasso (L1):** "Lasso = Diamond/Spiky → corners force weights to exactly zero → sparse → selection"

8.6 k-Means vs DBSCAN:

- "k-Means needs k, DBSCAN finds k"
- "k-Means finds spheres, DBSCAN finds shapes"
- "k-Means hates outliers, DBSCAN loves to find them"

8.7 PCA: "Eigenvectors point where data varies most, eigenvalues say how much"

8.8 Ensemble Mnemonic: "Bag → Parallel, Boost → Sequential, Stack → Hierarchical"

9 9. Common Interview Questions

9.1 Linear Regression

Q: What are the assumptions of linear regression and what happens when they're violated?

Model Answer: "There are 5 key assumptions (Gauss-Markov conditions):

1. **Linearity**: `E[y|X] = Xbeta`. Violation → biased predictions. Fix: add polynomial features, use trees.
2. **No multicollinearity**: features not perfectly correlated. Violation → unstable coefficients, wrong signs. Fix: Ridge regression, PCA, drop correlated features.
3. **Homoscedasticity**: constant error variance. Violation → inefficient OLS estimates, wrong standard errors. Fix: log-transform y, weighted least squares, robust SEs.
4. **No autocorrelation**: residuals uncorrelated. Violation in time-series → underestimated variance. Fix: ARIMA, GLS.
5. **Exogeneity**: errors uncorrelated with features. Violation → biased estimates. Fix: instrumental variables. For inference we also need normality of residuals, but OLS predictions remain unbiased without it."

Follow-up: "How would you detect heteroscedasticity?" "Plot residuals vs. fitted values — if variance increases with fitted values, it's heteroscedastic. Breusch-Pagan test formally."

Q: When would you use Ridge vs Lasso?

Model Answer: "Ridge (L2) when you believe all features are somewhat relevant — it shrinks all coefficients proportionally without zeroing any out. Good for multicollinearity — the added lambda*I term to `X^T X` ensures invertibility. Lasso (L1) when you want automatic feature selection — it produces sparse solutions (exact zeros) because the L1 ball has corners at the axes; the constraint boundary tends to intersect at corners. Better when true model is sparse. ElasticNet when you have correlated features AND want sparsity — Lasso tends to arbitrarily pick one feature from a correlated group; ElasticNet selects the group."

9.2 Decision Trees & Ensembles

Q: Why does Random Forest work? Why is it better than a single decision tree?

Model Answer: "Decision trees have **high variance** — small changes in training data produce very different trees. Random Forest exploits the bias-variance decomposition. If you average `B` trees with variance `sigma^2` and pairwise correlation `rho`, the ensemble variance is `rho*sigma^2 + (1-rho)/B * sigma^2`. By bootstrap sampling and random feature selection, we reduce `rho` (decorrelate the trees) without increasing bias. As `B` grows, the first term dominates — `rho*sigma^2` — which is why more trees always helps up to a point, but returns diminish once you've covered the correlation structure."

Follow-up: "What is OOB error? How is it different from cross-validation?" "Each bootstrap sample excludes ~36.8% of training data. These out-of-bag samples serve as a natural validation set for each tree. Average OOB error across trees estimates test error without needing a separate validation set. It's essentially built-in 'leave-~37%-out' cross-validation. The difference from k-fold CV: OOB uses the model trained on the bootstrap sample (not fold), so each tree is evaluated on ~37% of data rather than exactly 1/k."

Q: Explain gradient boosting from first principles.

Model Answer: "Gradient boosting is **gradient descent in function space**. We want to minimize some loss `L(y, F(x))`. Instead of optimizing weights `w` (as in neural networks), we optimize the function `F` itself.

Step 1: Start with a constant prediction: `F_0(x) = argmin sum L(y_i, c)`. Step 2: Compute pseudo-residuals = negative gradient of loss w.r.t. current predictions: `r_{im} = -[dL(y_i, F(x_i))/dF(x_i)]` For MSE, this is just `y_i - F(x_i)` — the regular residuals. Step 3: Fit a shallow tree to these pseudo-residuals. Step 4: Add this tree to the ensemble with a step size (learning rate). Step 5: Repeat.

The insight is that each tree is a **weak correction step** in function space. The sequence of trees collectively performs gradient descent on the loss function. XGBoost improves this by adding tree complexity regularization directly to the objective, approximating the optimal tree structure via a second-order Taylor expansion of the loss."

Follow-up: "What's the difference between XGBoost and LightGBM?" "Two key differences: (1) Tree growth strategy — XGBoost grows level-wise (all nodes at a depth before going deeper), LightGBM grows leaf-wise (always splits the leaf with max delta loss). Leaf-wise is more accurate with fewer trees but needs `num_leaves` control to avoid overfitting. (2) Data handling — LightGBM uses histogram binning for features (reduces memory and speeds up splits), GOSS sampling (keep high-gradient points, subsample low-gradient), and EFB feature bundling. Combined, LightGBM is 10-100x faster than XGBoost on large datasets."

9.3 SVM

Q: Explain the kernel trick. Why can't we just add polynomial features manually?

Model Answer: "The kernel trick lets us implicitly map data to a higher (or infinite) dimensional space without ever computing the feature vectors in that space.

Manually adding polynomial features: for degree-d polynomial on p features, you get `O(p^d)` features — for p=100, d=3, that's millions. This is computationally infeasible and causes memory issues.

The kernel trick works because the SVM dual formulation only depends on data through **inner products** `x_i^T x_j`. If we have a function `K(x_i, x_j) = phi(x_i)^T phi(x_j)` that computes the inner product in the high-dim space **directly**, we never need to compute `phi(x)` explicitly.

For the RBF/Gaussian kernel: `K(x,z) = exp(-gamma||x-z||^2)`, the implicit feature space is **infinite-dimensional**. We're essentially projecting to a space where any reasonable data can be linearly separated. But we never pay the computational cost of that infinite-dimensional space — kernel evaluation is O(p), not O(∞)."

Follow-up: "When would you NOT use an SVM?" "When n > ~50,000 — training is O(n^2) to O(n^3). When you need probability outputs (SVMs don't give them; Platt scaling is a post-hoc hack). When features are highly correlated and numerous. In these cases, gradient boosting or neural networks are better choices."

9.4 Clustering

Q: How do you choose k in k-Means? How do you evaluate clustering quality?

Model Answer: "For choosing k:

- **Elbow method**: Plot WCSS vs k; the 'elbow' is where adding more clusters has diminishing returns. Subjective and often ambiguous.
- **Silhouette score**: For each point, compute `(b - a)/max(a,b)` where `a` = mean intra-cluster distance, `b` = mean nearest-cluster distance. Range [-1,1], higher is better. Average across all points and choose k that maximizes it.
- **Gap statistic**: Compare WCSS to expected WCSS under null distribution (uniform random data). Choose k where the gap is largest.
- **Domain knowledge**: Often the most practical — business requirements define the number of segments.

For evaluation:

- **Internal metrics** (no ground truth): Silhouette, Davies-Bouldin index, Calinski-Harabasz
- **External metrics** (when ground truth exists): Adjusted Rand Index (ARI), Normalized Mutual Information (NMI), homogeneity, completeness"

9.5 PCA

Q: Explain PCA from first principles. Why is it useful?

Model Answer: "PCA finds directions of maximum variance in the data. Mathematically:

1. Center the data: `X_c = X - mean(X)`
2. Compute covariance matrix: `Sigma = (1/n) X_c^T X_c`
3. Eigen-decompose: `Sigma = V Lambda V^T` where V's columns are principal components
4. Project: `Z = X_c V_k` using the top k eigenvectors

The first PC is the direction that explains the most variance in the data. Second PC is orthogonal to first and explains the second most variance. By projecting onto top-k PCs, we retain most information in fewer dimensions.

Why useful:

- **Removes multicollinearity** — PCs are orthogonal by construction
- **Speeds up training** — fewer features
- **Denoises** — noise often lives in low-variance components
- **Visualization** — project to 2D/3D

Important caveats: PCA assumes variance = importance. If important features have low variance, PCA will discard them. Also, PCs are linear combinations of features — interpretability is lost. And PCA is scale-sensitive — always standardize features first."

9.6 Bias-Variance

Q: A model performs well on training data but poorly on test data. What's wrong, and how do you fix it?

Model Answer: "This is overfitting — high variance, low bias. The model has memorized training data including noise.

Diagnosis: large gap between train error and validation error.

Fixes in order of what to try:

1. **More data** (best fix — reduces variance directly)
2. **Regularization** (L1/L2 for linear models, max_depth/min_samples for trees, dropout for neural nets)
3. **Simpler model** (less expressive model family)
4. **Feature selection** (reduce dimensionality)
5. **Ensemble methods** (bagging reduces variance)
6. **Cross-validation** (ensure train/test aren't accidentally similar)
7. **Early stopping** (for iterative models)

If train error is also high: underfitting (high bias) — need more features, more complex model, less regularization."

10 10. Senior-Level Discussion Points

10.1 10.1 When to Choose Each Model in Production

The Reproducibility vs. Accuracy Tradeoff: A deep neural net may get 0.5% better AUC than XGBoost but requires 10x more engineering for monitoring, retraining, debugging, and serving. For production systems, the simpler model that the team can maintain is often better.

Feature engineering vs. architecture: In the era of AutoML and feature stores, the bottleneck has shifted from model complexity to feature quality. XGBoost with well-engineered features consistently beats neural networks on tabular data. The top Kaggle insight: feature engineering » model choice for tabular data.

10.2 10.2 Label Leakage and Data Pipeline Issues

More important than model choice: ensuring training/test splits don't leak future information. Time-based splits for time-series. Target encoding with cross-validation to prevent leakage. Calibration checks — log-loss can be low but probabilities systematically wrong (requires Platt scaling or isotonic regression).

10.3 10.3 Class Imbalance at Scale

At FAANG scale, class imbalance is extreme: CTR prediction has 0.1% positive rate, fraud detection 0.01%. Techniques:

- **Undersampling** with calibration correction (Facebook's negative downsampling + log-odds calibration)
- **Class weights** (`class_weight='balanced'` in sklearn)
- **Threshold tuning** — optimize at deployment time based on business costs
- **SMOTE** for small datasets only — generates synthetic samples
- **Focal loss** (Lin et al., RetinaNet) — down-weights easy negatives in loss function

10.4 10.4 Gradient Boosting vs Neural Networks for Tabular Data

Current consensus (Grinsztajn et al., 2022; Shwartz-Ziv & Armon, 2022): Tree-based models still dominate purely tabular tasks. Reasons:

- Tabular data is often "irregular" — features have different scales, types, and distributions; trees are invariant to monotone transformations
- Trees handle feature interactions naturally; neural nets need architectural tricks
- Trees don't suffer from uninformative features the same way

When neural nets win: Very large datasets (millions+ rows where boosting becomes slow), high-cardinality categoricals with learned embeddings, multi-modal inputs (tabular + text/image), need for transfer learning.

10.5 10.5 The Decomposition of Gradient Boosting Loss

The XGBoost objective uses a second-order Taylor expansion of the loss:

```python
L(y, F+h) ≈ L(y, F) + g*h + (1/2)*H*h^2

g = dL/dF    (gradient)
H = d^2L/dF^2   (Hessian)
```

This enables optimal closed-form leaf weights: `w_j* = -G_j / (H_j + lambda)` where G_j and H_j are gradient/Hessian sums in leaf j. The Hessian captures curvature — it's why XGBoost converges faster than first-order GBM.

10.6 10.6 Feature Importance Pitfalls

MDI (Mean Decrease in Impurity):

- Biased toward high-cardinality features (more possible splits = more likely to appear useful)
- Correlated features split importance between them, potentially understating each
- Use permutation importance or SHAP for production feature analysis

SHAP (SHapley Additive exPlanations):

- Based on game theory: feature importance = average marginal contribution across all feature coalitions

- TreeSHAP: O(T * L * D) — computationally feasible for trees
- Consistent and locally accurate by construction
- The gold standard for model explainability at FAANG

10.7 10.7 Hyperparameter Tuning Strategies

Method	When to Use
Grid Search	Small search space, deterministic needs
Random Search	Practical default — often finds good params faster than grid search (Bergstra & Bengio)
Bayesian Optimization (Optuna, Hyperopt)	Expensive models, important to find optimum
Early Stopping + CV	Tree models — add trees until validation error stops improving
Population-Based Training	Parallel GPU training at Google DeepMind scale

10.8 10.8 Calibration

A model with good AUC can have terrible calibration. In ads, you need calibrated probabilities for expected value calculation: `E[value] = P(click) * bid`. If P(click) is systematically 2x too high, you overbid and lose money.

Calibration methods:

- **Platt scaling:** Fit sigmoid on top of model output
- **Isotonic regression:** Non-parametric, more flexible
- **Temperature scaling:** For neural networks

Reliability diagram: Plot predicted probability bins vs actual event rate. Perfect calibration = diagonal line.

11 11. Typical Mistakes Candidates Make

11.1 Conceptual Mistakes

1. **"Higher k in k-NN reduces overfitting"** — Partially right: higher k reduces variance, but increases bias. The optimal k balances both.
2. **"Decision trees are always interpretable"** — Only shallow trees (depth 3-4). A depth-20 tree with 1M nodes is as uninterpretable as a neural network.
3. **"SVM kernel picks the right feature space automatically"** — The kernel defines the feature space; choosing the wrong kernel (e.g., RBF when data is linearly separable) can hurt performance.
4. **"Gradient boosting doesn't overfit"** — It absolutely can. More trees + high learning rate = overfit. Always use early stopping on a validation set.
5. **"PCA removes unimportant features"** — PCA removes LOW-VARIANCE directions. Low variance $\neq$ unimportant. If an important feature has low variance (binary label), PCA may remove it.
6. **"Logistic regression assumes the output is normally distributed"** — No, it assumes the **log-odds** are linear in features. The output is a Bernoulli random variable.
7. **"Random Forest is just bagging"** — Random Forest adds feature subsampling at each split (not just bootstrap rows). This is the critical innovation that decorrelates trees further.

11.2 Implementation Mistakes

8. **Not standardizing features for distance-based models** — k-NN, SVM, PCA, k-Means all need feature scaling. A feature with range [0,1000] will dominate one with range [0,1].
9. **Using test set to select k in k-NN** — Data leakage. Use nested cross-validation or a separate validation set.
10. **Target encoding without cross-validation** — Encoding target mean per category on full training set leaks label information. Use K-fold target encoding.
11. **Not handling class imbalance** — Using accuracy as metric on 99/1 split (always predict majority → 99% accuracy, useless model). Use AUC, F1, or precision-recall.
12. **Comparing models on training accuracy** — Always compare on held-out validation/test data. Training accuracy is only useful for diagnosing underfitting.
13. **k-Means++ initialization vs. random** — Default in sklearn is k-means++. Saying "k-Means is sensitive to initialization" without mentioning k-means++ shows lack of depth.

11.3 Interview-Specific Mistakes

14. **Not asking clarifying questions** — In a system design or ML design interview, jumping to XGBoost without asking about data size, latency requirements, interpretability needs shows poor engineering judgment.
15. **Ignoring data size constraints** — SVM is O(n^3) — saying "I'd use SVM" for a billion-row dataset is a red flag.
16. **Not connecting to bias-variance** — When asked about regularization, hyperparameters, or overfitting, always frame the answer in terms of the bias-variance tradeoff.

12 12. How This Connects To Other Topics

12.1 ML Fundamentals

- **Bias-Variance Tradeoff**: Trees (high variance) → Random Forest (low variance) → Understanding this is the foundation for hyperparameter tuning
- **Optimization**: GD for logistic regression → Adam for neural nets → FTRL for online learning
- **Feature engineering**: Classic models expose which features matter → informs deep learning feature design
- **Cross-validation**: Model selection for ALL algorithms; k-fold, stratified, time-series CV

12.2 Deep Learning

- **Logistic Regression → Neural Nets**: Logistic regression = neural net with no hidden layers. Adding hidden layers = deep learning.
- **Gradient Boosting → DNNs**: GBDT-NN hybrid (Meta's approach) bridges the gap. DNNs learn representations; GBDTs learn feature interactions on tabular data.
- **Kernels → Radial Basis Function Networks**: RBF kernel SVMs are equivalent to shallow RBF networks.
- **PCA → Autoencoders**: Linear PCA is the special case of a linear autoencoder with identity activation. Non-linear autoencoders = kernel PCA generalization.
- **Embeddings + k-NN**: Neural nets generate embeddings, k-NN (with ANN indexing) retrieves nearest neighbors — this is the two-tower model architecture used in production recommendations.

12.3 System Design

- **Inference latency**: Linear model: sub-millisecond. k-NN (naive): O(n*p*) — *slow for large n. FAISS approximate k-NN: O(log n). XGBoost: O(M*depth) — typically fast.
- **Training throughput**: Random Forest is embarrassingly parallel. Boosting is sequential. This affects distributed training architecture choices.
- **Model size**: SVM stores support vectors (can be large); RF stores all trees (large); Logistic Regression = one weight vector (tiny).
- **Online learning**: Logistic Regression with SGD supports online updates. Trees and SVMs do not (need retraining). Important for streaming ML systems.
- **Feature stores**: Classic models require explicit feature engineering → motivates centralized feature computation and caching infrastructure.
- **A/B testing**: Statistical models (logistic regression) give p-values; gradient boosting requires bootstrap confidence intervals — affects significance testing in experiments.

13 13. FAANG Interview Tips

13.1 The ML Design Interview Framework

When asked "design an ML system for X":

1. **Clarify** — What's the objective metric? What's the data size? Latency requirements? Interpretability needed?
2. **Baseline** — Always start with Logistic Regression or a simple GBM as baseline
3. **Features** — Spend most time here; features matter more than model choice
4. **Model selection** — Justify based on data size, latency, interpretability, expected nonlinearity
5. **Evaluation** — Offline metrics (AUC, NDCG) vs. online metrics (CTR, conversion)
6. **Production concerns** — Distribution shift, retraining frequency, monitoring

13.2 Coding Interviews

You may be asked to implement from scratch:

Python

```python
# Linear Regression (gradient descent)
def fit(X, y, lr=0.01, epochs=1000):
    n, p = X.shape
    w = np.zeros(p)
    for _ in range(epochs):
        y_hat = X @ w
        grad = (2/n) * X.T @ (y_hat - y)
        w -= lr * grad
    return w

# k-NN Classification
def predict(X_train, y_train, x_test, k=5):
    distances = np.sqrt(np.sum((X_train - x_test)**2, axis=1))
    k_indices = np.argsort(distances)[:k]
    k_labels = y_train[k_indices]
    return np.bincount(k_labels).argmax()

# k-Means
def kmeans(X, k, max_iter=100):
    centroids = X[np.random.choice(len(X), k, replace=False)]
```

```
    for _ in range(max_iter):
        # Assignment
        dists = np.sqrt(((X - centroids[:, np.newaxis])**2).sum(axis=2))
        labels = np.argmin(dists, axis=0)
        # Update
        new_centroids = np.array([X[labels==i].mean(0) for i in range(k)])
        if np.allclose(centroids, new_centroids):
            break
        centroids = new_centroids
    return labels, centroids

# Logistic Regression
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def fit_logistic(X, y, lr=0.1, epochs=1000):
    n, p = X.shape
    w = np.zeros(p)
    for _ in range(epochs):
        p_hat = sigmoid(X @ w)
        grad = (1/n) * X.T @ (p_hat - y)
        w -= lr * grad
    return w
```

13.3 Typical FAANG Question Flows

"Which model would you use for CTR prediction at ads scale?" → "Start with Logistic Regression for speed and interpretability (FTRL for online learning). Move to GBDT for capturing feature interactions (XGBoost or LightGBM). At scale, use a hybrid: GBDT to generate crossed features fed into a neural net (like Meta's DLRM). Key considerations: training data is billions of rows, serving latency < 5ms, need calibrated probabilities for expected value computation."

"How does XGBoost handle missing values?" → "During training, XGBoost learns a default direction for missing values — for each split, it tries routing missing values to left vs. right and picks whichever reduces the loss more. At inference, missing values follow the learned default direction. This is learned automatically, not imputed."

"Why does bagging work but independent models don't?" → "Independence is the key. Training independent models on the same dataset gives correlated predictions — their errors don't cancel. Bootstrap sampling + random feature subsampling reduces the correlation between trees' errors. The variance formula shows this clearly: `Var(avg) = rho * sigma^2 + (1-rho)/B * sigma^2`. Independent models have `rho=0` but you'd need separate data. Bootstrap gives `rho>0` but real data, which is the practical trade-off."

14 14. Revision Cheat Sheet

14.1 10-Minute Summary

Linear Regression: Fit line by minimizing squared errors. OLS = `w = (X^T X)^{-1} X^T y`. Ridge: L2 reg, no zeros. Lasso: L1 reg, sparsity. Assumptions: linearity, no multicollinearity, homoscedasticity.

Logistic Regression: `P(y=1|x) = sigmoid(w^T x)`. Loss = log-loss (cross-entropy). No closed form. Linear boundary. Coefficients = log-odds change per unit feature.

Decision Tree: Recursively split on best feature (Gini or IG). Greedy, high variance, interpretable. Prune with max_depth, min_samples_leaf. CART algorithm.

Random Forest: B trees on bootstrap samples + random feature subset at each split. Re-

duces variance without increasing bias. OOB error = free validation. Feature importance via MDI or permutation.

Gradient Boosting: Sequential trees fitting pseudo-residuals (negative gradient). XGBoost adds L2 tree regularization + second-order Taylor + sparse-aware. LightGBM adds leaf-wise growth + histograms + GOSS + EFB. Fastest on tabular data.

SVM: Maximize margin hyperplane. Support vectors define it. Soft margin: C trades margin width vs. violations. Kernel trick: implicit high-dim mapping via inner products. RBF = infinite-dim, O(p) computation.

k-NN: Store all training data. Predict by k-nearest votes/mean. Lazy learning. Curse of dimensionality. Need feature scaling. k-means++ init. Use ANN (FAISS) for scale.

Naive Bayes: `P(y|x) ∝ P(y) * prod P(x_j|y)`. Assumes conditional independence. Fast, works on text. Gaussian/Multinomial/Bernoulli variants. Laplace smoothing for zero probabilities.

k-Means: Assign points to nearest centroid, update centroids as mean. WCSS objective, non-convex. k-means++ init. Elbow/silhouette for choosing k. Spherical clusters assumption.

Hierarchical: Build dendrogram bottom-up (agglomerative). Ward linkage = minimize variance increase. Cut dendrogram to get k clusters. O(n^2) memory.

DBSCAN: Core/border/noise points. Finds arbitrary shapes. No need to specify k. Sensitive to eps and min_samples. Natural outlier detection.

PCA: Eigenvectors of covariance matrix = principal components. Project onto top-k for dimensionality reduction. Use SVD in practice. Standardize first. Explained variance ratio for choosing k.

Ensembles: Bagging = parallel, reduces variance. Boosting = sequential, reduces bias. Stacking = meta-model on predictions.

14.2 Algorithm Decision Table --- "Which Algorithm When"

Scenario	Recommended	Why
Tabular data, maximize accuracy	LightGBM / XGBoost	State-of-the-art on tabular; handles all feature types
Need interpretability / regulatory compliance	Logistic Regression or Decision Tree (shallow)	Explicit feature coefficients or printable rules
Baseline model	Logistic Regression	Fast, calibrated, easy to improve upon
High-dimensional sparse features (text/NLP)	Logistic Regression (L1/L2) or Linear SVM	Efficient on sparse data; SVM with linear kernel is L2-LR
Small dataset (n < 1000), many features	SVM (RBF kernel)	Effective in high p/n ratio; global optimum
Non-linear patterns, medium dataset	SVM + RBF kernel or Random Forest	Kernel SVM for structured non-linearity; RF for robustness
Very large dataset (n > 1M), need speed	LightGBM	Fastest tree-based; designed for large-scale
Text classification, real-time	Naive Bayes	Sub-millisecond inference; surprisingly competitive
Need calibrated probabilities	Logistic Regression or calibrated GBM	LR is naturally calibrated; GBM + Platt scaling
Missing values, heterogeneous features	XGBoost / LightGBM	Native missing value handling; no preprocessing
Unknown number of clusters, outlier detection	DBSCAN	Finds k automatically; labels noise points

Scenario	Recommended	Why
Known k, spherical clusters	k-Means	Fast, interpretable, works well with good init
Cluster hierarchy / exploratory analysis	Hierarchical (Ward linkage)	Dendrogram shows structure at all levels
Nearest neighbor retrieval at scale	FAISS / HNSW (ANN)	Exact k-NN is O(n*p); ANN is O(log n)
Recommendation (item-item, user-item)	k-NN in embedding space	After generating embeddings with Matrix Factorization/DNN
Remove multicollinearity before linear model	PCA	Creates orthogonal features
Visualization of high-dim data	t-SNE / UMAP	Preserves local/global structure; PCA first for large n
Feature compression (autoencoder alternative)	PCA	Linear compression; interpretable explained variance
Online/streaming learning	Logistic Regression (SGD)	Tree models can't update incrementally
Maximum accuracy, no latency constraint	Stacking (GBM + RF + LR meta)	Combines diverse model predictions
Interpretable feature interactions	Decision Tree → tree.plot()	Explicit IF-THEN rules for business users
Imbalanced classes, small minority	XGBoost + class_weight + threshold tuning	Robust to imbalance; easy threshold calibration
Anomaly detection (labeled)	Logistic Regression / Isolation Forest	LR if labeled; IF for unsupervised
Anomaly detection (unlabeled)	DBSCAN / Isolation Forest / Autoencoder	DBSCAN: density-based; IF: tree-based isolation

14.3 Complexity Quick Reference

Algorithm	Train Time	Inference Time	Memory
Linear Regression	O(n*p^2) closed form	O(p)	O(p)
Logistic Regression	O(n*p*iter)	O(p)	O(p)
Decision Tree	O(n*p*log^2 n)	O(depth)	O(n)
Random Forest	O(B*n*sqrt(p)*log^2 n)	O(B*depth)	O(B*n)
XGBoost	O(M*n*p*log n)	O(M*depth)	O(n*p)
LightGBM	O(M*n*255*log n)	O(M*depth)	O(n*255)
SVM	O(n^2 p to n^3 p)	O(n_sv * p)	O(n_sv)
k-NN	O(n*p) store	O(n*p + n log n) per query	O(n*p)
Naive Bayes	O(n*p)	O(p*K)	O(p*K)
k-Means	O(n*k*p*iter)	O(k*p)	O(n*k)
Hierarchical	O(n^2 log n)	O(1) post	O(n^2)
DBSCAN	O(n log n) with index	O(1)	O(n)
PCA	O(n*p*k) randomized	O(p*k)	O(p*k)

14.4 Key Formulas Cheat Sheet

```
LOSS FUNCTIONS:
MSE (regression):     L = (1/n) * sum[(y_i - y_hat_i)^2]
Log-loss (binary):    L = -(1/n) * sum[y*log(p) + (1-y)*log(1-p)]
Hinge (SVM):          L = (1/n) * sum[max(0, 1 - y_i * f(x_i))]

IMPURITY MEASURES:
Gini:     G = 1 - sum_k[p_k^2]          (0 = pure, 0.5 = worst binary)
Entropy:  H = -sum_k[p_k * log2(p_k)]   (0 = pure, 1 = worst binary)

REGULARIZATION:
Ridge: L = MSE + lambda * sum(w_j^2)
Lasso: L = MSE + lambda * sum(|w_j|)
SVM:   min (1/2)||w||^2 + C * sum[xi_i]

PROBABILITY:
Sigmoid:  sigma(z) = 1/(1 + e^{-z})
Softmax:  P(y=k) = e^{w_k^T x} / sum_j[e^{w_j^T x}]
Bayes:    P(y|x) ∝ P(x|y) * P(y)

KERNELS:
Linear:   K(x,z) = x^T z
RBF:      K(x,z) = exp(-gamma * ||x-z||^2)
Poly:     K(x,z) = (gamma * x^T z + r)^d

DIMENSIONALITY:
PCA:      Sigma v = lambda v    (eigenvectors of covariance)
EVR:      EVR_k = lambda_k / sum_i lambda_i

VARIANCE OF ENSEMBLE:
Var(avg B corr trees) = rho*sigma^2 + (1-rho)*sigma^2/B
```

14.5 Most Important Concepts --- Final 5

1. **Gradient boosting = gradient descent in function space** — Each tree corrects the residual error of the ensemble so far; sequential trees are steps along the loss gradient.
2. **Kernel trick = inner product without feature expansion** — `K(x,z) = <phi(x), phi(z)>` computed in O(p) without computing phi, enabling infinite-dimensional feature spaces.
3. **Bagging reduces variance by decorrelating trees; boosting reduces bias by sequential correction** — Know these two error types, know which ensemble addresses each.
4. **Bias-Variance Decomposition frames everything** — Every hyperparameter choice (tree depth, k in k-NN, C in SVM, lambda in regularization) is trading off bias vs. variance. Always frame your answers through this lens.
5. **The "right algorithm" depends on constraints** — Data size (n, p), latency budget, interpretability requirements, missing values, class balance. There is no universally best algorithm — demonstrating this judgment is what FAANG interviewers want.

Word count: ~9,800 words | Section count: 14 (Overview through Revision Cheat Sheet)

15 Visual Reference

A set of figures distilling the key quantitative intuitions of this topic.

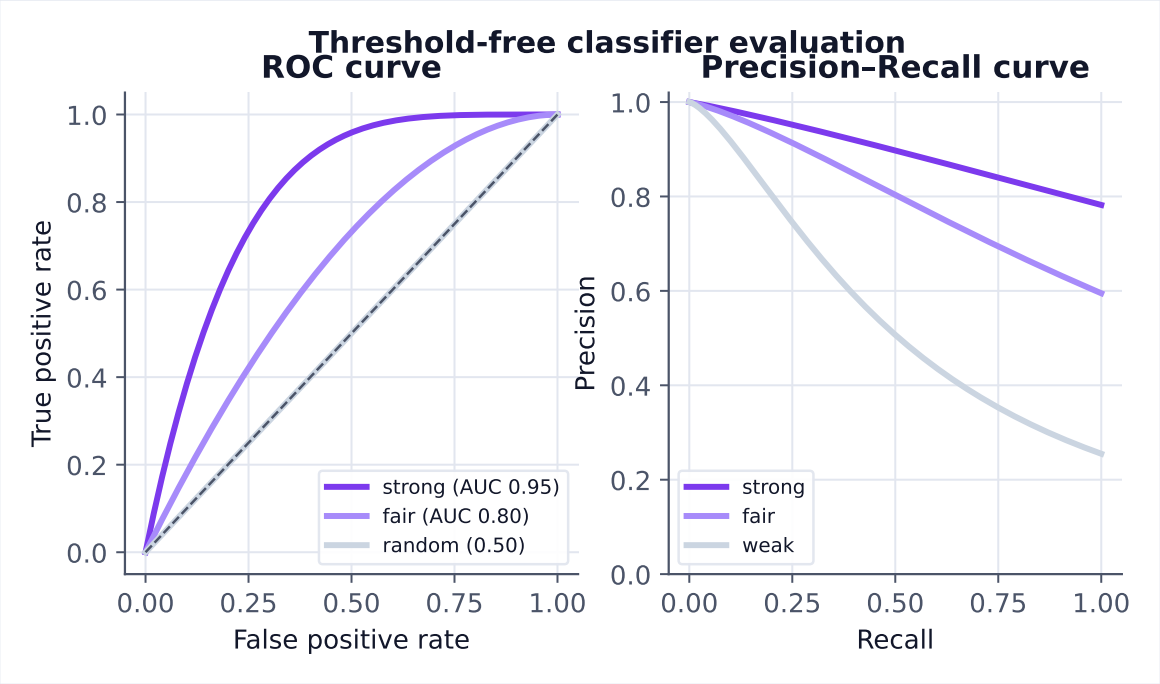


Figure 1. ROC summarizes ranking quality across thresholds; PR is more informative under heavy class imbalance, where a high AUC can still hide poor precision.

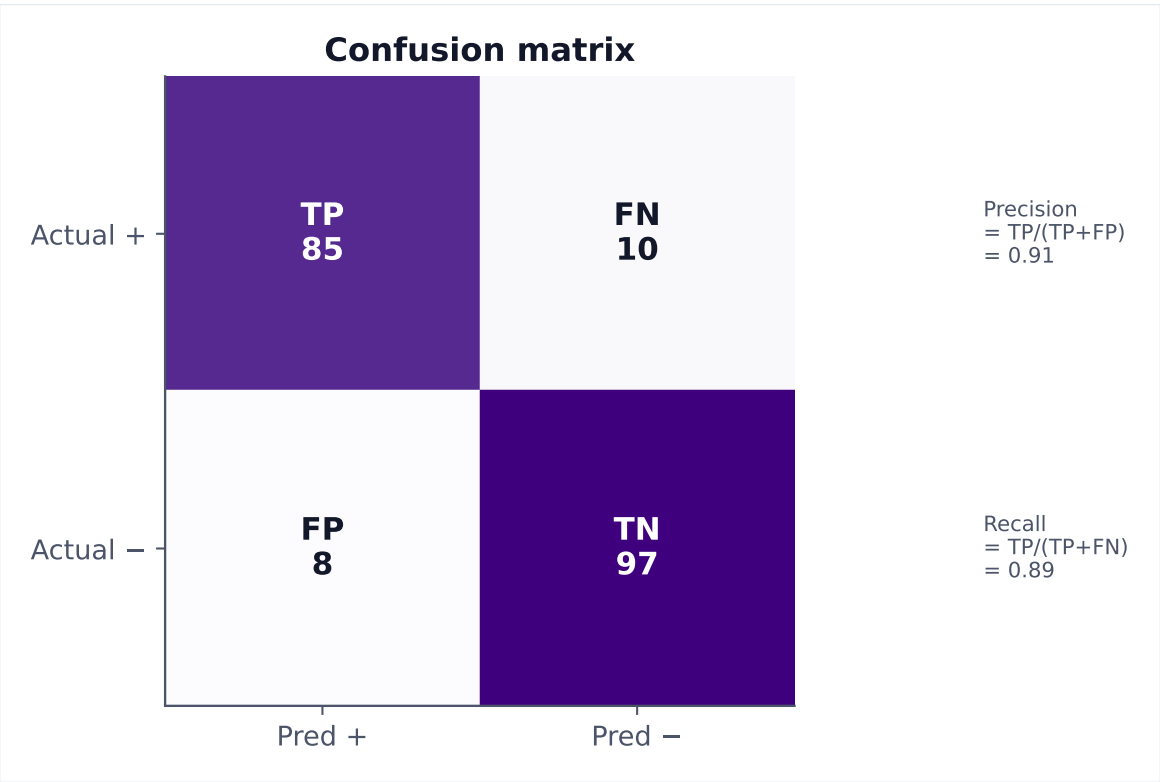


Figure 2. Every classifier error is a false positive or false negative. Precision penalizes FPs, recall penalizes FNs — which matters more depends on the cost of each mistake.